

Guia Dispositiu Arduino

Guia per desenvolupar el lector de targetes i enviar les dades al servidor.





Passos del desenvolupament.

Passos:

1. Llegir una targeta amb arduino, i mostrar-la pel port sèrie.
2. Configurar la wifi i comprovar internet.
3. Crear un dispositiu IoT en Amazon Web Services que representi el nostre Arduino (node01).
4. Connectar el dispositiu Arduino a AWS i enviar les dades de la targeta.
5. Exemple programa principal.
6. Crear un segon dispositiu en AWS que representi el programa que rebrà les dades (client01).
7. Programar el client per a que es connecti a AWS i reculli les dades de la targeta.
8. Programar el client per a que desi les dades de la targeta en una base de dades.



1-Llegir una targeta amb arduino

Per llegir una targeta de proximitat amb Arduino esp32, que és el vostre model, heu d'utilitzar la llibreria que està assenyalada en la imatge.

Quan instal·leu una llibreria, s'afegeixen també exemples, que teniu al menú File->Examples. Al final hi ha una llista de les llibreries instal·lades. Obrir l'exemple que posa ReadNUID.

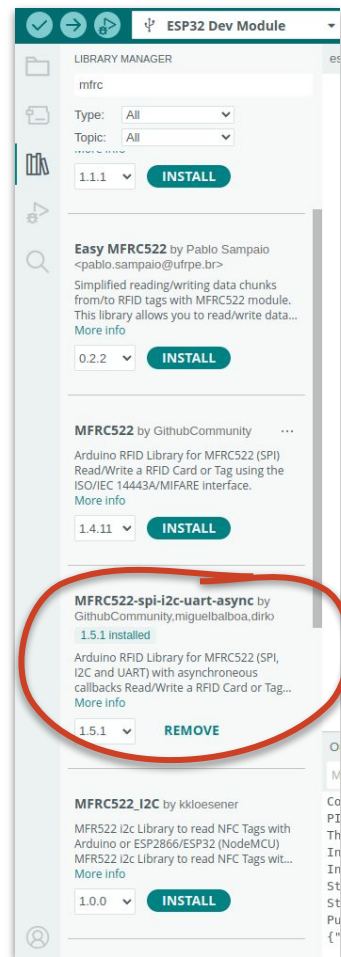
Per al esp32, els pins seleccionables han de ser:

```
#define SS_PIN 5  
#define RST_PIN 0
```

tal i com indiquen les instruccions que trobareu en aquesta adreça:

<https://www.electronicwings.com/esp32/rfid-rc522-interfacing-with-esp32>

**Compte, en la pàgina indica una altra llibreria. Vosaltres heu d'utilitzar la que hem indicat.*





1-Llegir una targeta amb arduino

Quan funcioni la lectura amb l'exemple, us suggereixo crear un fitxer a part per a les funcions de lectura, i deixar el programa principal amb el mínim codi. Afegir arxius addicionals no suposa cap diferència per al compilador. Ho tracta com si tot el codi estigués en un sol arxiu.

Allà podreu incloure la llibreria, i realitzar les funcions pròpies del dispositiu MRFC522.:

Recomenació de codi:

```
#include <SPI.h>
#include <MFRC522.h>

#define SS_PIN 5
#define RST_PIN 0
MFRC522 rfid(SS_PIN, RST_PIN);

void SetupRFID ()
{
  //Aqui ha d'anar el setup inicial.
  //Aquesta funcio la crides des del Setup() principal.
}

bool CheckRFID(String &tag){
  //Aqui ha d'anar el codi que detecta si hi ha una targeta.
  //Si hi ha una targeta, retorna cert.
  //el paràmetre tag contindrà el Id de la targeta
}
```



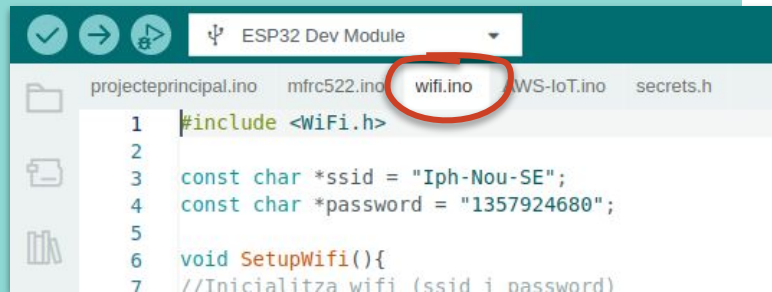


2-Configurar la wifi

Per configurar la wifi, busqueu l'exemple: File->Examples->Wifi->WifiClient

Altra vegada, recomanem implementar les funcions en un arxiu separat:

```
#include <WiFi.h>
const char *ssid = "Android de David";
const char *password = "1234567890";
void SetupWifi() {
//Inicialitza wifi (ssid i password)
}
void CheckWifi() {
//Si no esta connectat, connecta..
}
```



Important: **NO utilitzeu** cap exemple que inclogui WifiMulti.h. Aquest fitxer serveix per afegir diferents ssid, però provoca un error i fa que l'Arduino es reiniciï contínuament. Si us passa, haureu de realitzar un procediment per esborrar el programa defectuós.



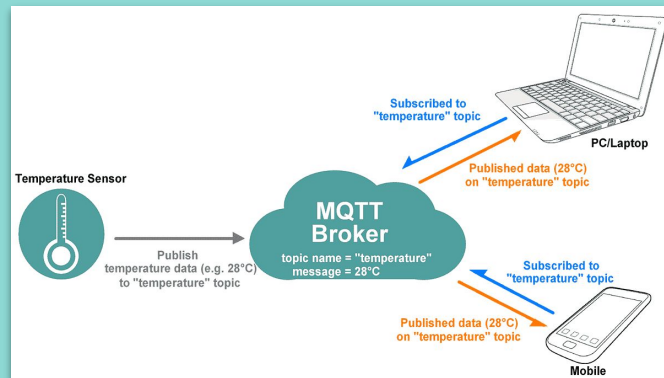
3-Crear un dispositiu IoT

Per crear un dispositiu IoT en Amazon Web Services que representi el nostre Arduino (node01), necessitem saber una mica sobre AWS i MQTT:

- 3.1- Introducció a AWS IoT
- 3.2-Protocol MQTT
- 3.3-Crear un dispositiu IoT en AWS

Una mica d'informació addicional sobre AWS-IOT i el protocol MQTT:

- <https://aws.amazon.com/what-is/mqtt/>
- <https://diyiOt.com/introduction-into-mqtt/>





3.1- Introducció a AWS IoT

- Amazon Web Services és un conjunt de serveis al núvol, com per exemple màquines virtuals, xarxes virtuals, i també un apartat IoT-Core.
- Aquest servei és una implementació del protocol MQTT, abastament utilitzat en la indústria d'Internet de les Coses.
- Utilitzarem aquest servei com a intermediari entre el nostre dispositiu Arduino, i el servidor de bases de dades.
- A continuació una mica d'informació sobre aquest protocol.



3.2-Protocol MQTT

El protocol **MQTT (Message Queuing Telemetry Transport)** és un protocol de comunicació lleuger basat en publicació/subscripció que s'utilitza principalment en aplicacions d'Internet de les Coses (IoT) per enviar missatges entre dispositius amb una mínima càrrega de xarxa. Aquí tens un resum del seu funcionament:

Característiques principals de MQTT:

Model Publicació/Subscripció:

- **Publicador:** Un dispositiu que envia missatges.
- **Subscriber:** Un dispositiu que rep missatges.
- **Broker:** Un servidor intermediari que gestiona els missatges entre els publicadors i subscriptors.

Els publicadors envien missatges a un "topic" específic, i els subscriptors s'inscriuen als topics d'interès. El broker s'encarrega de lliurar els missatges dels publicadors als subscriptors que estiguin inscrits en el topic corresponent.

Topics: Un topic és una cadena de text jeràrquica que identifica categories de missatges. Els subscriptors poden escoltar diferents topics (com per exemple, "casa/sala/temperatura").



3.2- Protocol MQTT

Flux de funcionament:

1. **Connexió:** El client MQTT es connecta al broker, passant les credencials si és necessari.
2. **Subscripció:** Un client pot subscriure's a un topic o diversos per rebre missatges.
3. **Publicació:** Un client pot publicar un missatge en un topic, i el broker enviarà aquest missatge a tots els clients que estiguin subscrits a aquest topic.
4. **Desconnexió:** Quan un client ja no necessita més informació, es pot desconnectar del broker.

Exemple senzill:

1. Un sensor de temperatura (publicador) envia dades cada 10 segons al topic `casa/sala/temperatura`.
2. Un telèfon mòbil (subscriptor) està subscrit a aquest topic. El broker MQTT rep els missatges del sensor i els redirigeix al telèfon.
3. Si el sensor es desconnecta inesperadament, el broker pot enviar un missatge de "Last Will" a tots els subscriptors indicant que el sensor ha deixat de funcionar.



3.3-Crear un dispositiu IoT en AWS

Ara crearem un dispositiu en AWS.

El servei s'anomena IoT-core, i bàsicament el que farem serà crear unes claus per a que el nostre Arduino s'hi pugui connectar, i així donar un poc de seguretat a la nostra infraestructura.

Si no ho féssim, qualsevol que conegués la nostra IP d'AWS podria enviar o rebre dades.

En AWS IoT crearem un “Objeto” o “Thing”, depenen si ho tenim en castella o anglés, l'associarem uns permisos i certificats, que utilitzarem des d'arduino.

A continuació teniu un manual de com crear el “Objeto”, i utilitzar-lo des d'arduino per enviar dades (publicar).



3.3-Crear un dispositiu IoT en AWS

Entra en AWS amb el teu compte que tindràs en Amazon Web Services. (email del centre).

Segueix les instruccions per crear un dispositiu (en anglès una “Thing”), en l’apartat IoT-Core, tal com s’explica en la següent pàgina:

<https://medium.com/caleidos/domina-el-iot-desde-cero-controla-una-luz-led-con-esp32-y-aws-iot-core-ac19033a039b>



4-Connectar el dispositiu Arduino a AWS

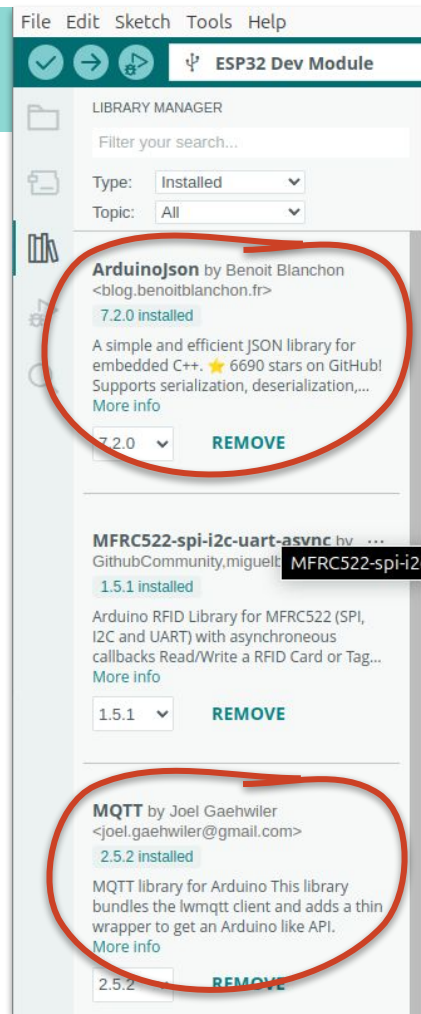
Segueix les instruccions de la pàgina anterior, per crear un programa de prova amb Arduino.*

Les llibreries són les que s'indiquen a la imatge.

Aquest programa en realitat rep dades des d'AWS, però consulta amb els exemples que venen en la llibreria, com enviar dades cap a AWS IoT.

Comprova que pots rebre dades des de AWS, i que, per tant, estàs subscrit correctament.

**Compte, en la pàgina web es diu de crear un arxiu secrets.h amb la informació dels certificats, però erròniament el nom que proposa és secrets.sh (l'extensió ha de ser .h per a poder fer l'include després. !!!)*





4-Connectar el dispositiu Arduino a AWS

Un cop hakis provat la connexió amb AWS i hakis provat d'enviar i rebre missatges, és l'hora d'integrar-ho en el projecte principal.

Crea **dos** nous arxius al projecte (AWS-IoT, i secrets.h).

- L'arxiu **secrets.h** contindrà els certificats.
- L'arxiu **AWS-IoT** contindrà les funcions d'accés a AWS
 - SetupAWS() //Inicialitza AWS
 - CheckAWS() //si no està connectat, es connecta
 - PublicaTAG(String tagID) //Envia el id de targeta al servidor AWS



4-Exemple codi AWS

Arxiu secrets.h

```
#include <pgmspace.h>

const char WIFI_SSID[] = "";
const char WIFI_PASSWORD[] = "";
const char AWS_IOT_ENDPOINT[] = "xxxxxxxxxxxx-ats.iot.us-east-1.amazonaws.com" ;

// Amazon Root CA 1
static const char AWS_CERT_CA[] PROGMEM = R"EOF(
-----BEGIN CERTIFICATE-----
-----END CERTIFICATE-----
)EOF";

// Device Certificate
static const char AWS_CERT_CRT[] PROGMEM = R"KEY(
-----BEGIN CERTIFICATE-----
-----END CERTIFICATE-----
)KEY";

// Device Private Key
static const char AWS_CERT_PRIVATE[] PROGMEM = R"KEY(
-----BEGIN RSA PRIVATE KEY-----
-----END RSA PRIVATE KEY-----
)KEY";
```





4-Exemple codi AWS

Arxiu AWS-IoT

```
#include "secrets.h"
#include <WiFiClientSecure.h>
#include <MQTTClient.h>
#include <ArduinoJson.h>

#define THINGNAME "espnode01"
// The MQTT topics that this device should publish/subscribe
//Compte, no pot començar per /
#define AWS_IOT_SUBSCRIBE_TOPIC "iticbcn/espnode01/sub"
#define AWS_IOT_PUBLISH_TOPIC "iticbcn/espnode01/pub"

WiFiClientSecure net =WiFiClientSecure();
MQTTClient client =MQTTClient(256);
```

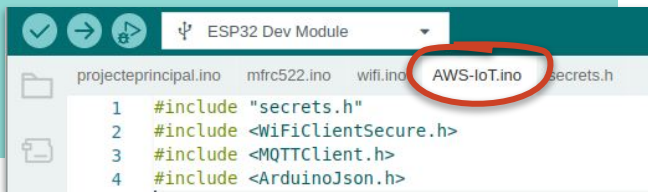
```
void SetupAWS () {
//Actualitzar la variable d'estat AWSConnected
}

void CheckAWS () {

    if (!client.connected()) {
        //Connectar-se a AWS.
        //Comprovar Si està connectat
        //Actualitzar la variable d'estat AWSConnected
        //Subscriure al topic

    }else{
        //Actualitzar la variable d'estat AWSConnected
        client.loop();
    }
}

void PublicaTag (String tagID){
//un cop tenim el tag en un string el podem publicar a AWS
// el topic pot ser quelcom com "iticbcn/node01/pub"
}
```





5-Exemple programa principal

```
//Variables d'estat
bool WifiConnected=false;
bool AWSConnected=false;

void setup() {

    // put your setup code here, to run once:
    SetupWifi();
    SetupRFID();
    SetupAWS();

}

void loop() {
    // put your main code here, to run repeatedly:
    CheckWifi();
    //Si estem connectats a wifi, connectem a AWS.
    //Si estem connectats a AWS mirem si hi ha tarja per llegir
    //Si estem connectats a AWS i hi ha tarja per llegir,
    //Publica Tag
}
```





6-Crear un segon dispositiu en AWS

Pots crear un segon dispositiu (thing) a AWS per connectar-te des d'un altre programa

Repeteix les instruccions anteriors per crear un segon dispositiu en AWS.

Aquest altre dispositiu tindrà altres certificats.

Els hauràs d'utilitzar per connectar-te des d'algun programa fet des del servidor. (mira el tema següent)



7 i 8-Programar el client amb AWS

Programa un client que es connecti a AWS-IoT, rebí les dades de les targetes.

Es pot fer amb Java (Versió 1 del SDK, més senzilla):

<https://github.com/aws/aws-iot-device-sdk-java>

En bash, python i javascript, i java :

<https://docs.aws.amazon.com/iot/latest/developerguide/mqtt.html>

També es pot fer en Node.js o Node-Red

Utilitzeu servidors a Lambda o EC2 de AWS, per al client de AWS .

Quan comproveu que podeu rebre les dades, modifiqueu el programa perquè ho guardi en una base de dades.